Ok, but what about predictive AI?

We are here to talk about the AI that pays the bills.

Who We Are!

BRAYAN MUÑOZ V.

SWE, JUG Leader

𝕏 @brayanmnz_

J. RAFAEL ALMONTE

SWE, JUG Leader

𝕏@jralmontec

Disclaimer

Everything we are sharing today comes from our own hands-on experience and research. We aren't part of, sponsored by, or affiliated with any of the companies or tools we'll be discussing, we're just sharing what we've learned from using them.

The State of AI in 2026

Generative AI

Predictive AI

The State of AI in 2026

Generative AI

- Creative

Predictive AI

The State of AI in 2026

30% of GenAI projects will be abandoned due to a lack of clear business value. [3]

Gener

- Creative

Predictive AI

[1] McKinsey & Company, "The state of AI in 2025: Agents of innovation," McKinsey Global Institute, 2025.

[2] McKinsey & Company, "The state of AI: How organizations are rewiring to capture value," McKinsey Global Institute.

[3] Deloitte, "State of AI 2026 Global," Deloitte AI Institute, 2026.

The State of AI in 2026

Generative AI

- Creative
- Costly

Predictive AI

The State of AI in 2026

Generati

- C
- Costly

75% of leaders say infrastructure and scaling costs are a major barrier. [3]

Predictive AI

[1] McKinsey & Company, "The state of AI in 2025: Agents of innovation," McKinsey Global Institute, 2025.

[2] McKinsey & Company, "The state of AI: How organizations are rewiring to capture value," McKinsey Global Institute.

[3] Deloitte, "State of AI 2026 Global," Deloitte AI Institute, 2026.

The State of AI in 2026

Generative AI

- Creative
- Costly
- Hype

Predictive AI

The State of AI in 2026

Generative AI

- C
-
- Hype ?

Only ~6% of companies have achieved an EBIT increase of over 5% with GenAI. [1]

Predictive AI

[1] McKinsey & Company, "The state of AI in 2025: Agents of innovation," McKinsey Global Institute, 2025.

[2] McKinsey & Company, "The state of AI: How organizations are rewiring to capture value," McKinsey Global Institute.

[3] Deloitte, "State of AI 2026 Global," Deloitte AI Institute, 2026.

The State of AI in 2026

Generative AI

- Creative
- Costly
- Hype ?

Predictive AI

- Deterministic

The State of AI in 2026

Zero hallucinations. (Vs. 47% of GenAI users reporting data inaccuracy/leakage issues). [3]

Generative AI

- Creative
- Costly
- Hype ?

Predictive AI

- Deterministic

[1] McKinsey & Company, "The state of AI in 2025: Agents of innovation," McKinsey Global Institute, 2025.

[2] McKinsey & Company, "The state of AI: How organizations are rewiring to capture value," McKinsey Global Institute.

[3] Deloitte, "State of AI 2026 Global," Deloitte AI Institute, 2026.

The State of AI in 2026

Generative AI

- Creative
- Costly
- Hype ?

Predictive AI

- Deterministic
- Profitable

The State of AI in 2026

AI leaders attribute over 20% of their net earnings (EBIT) to Analytical/Predictive AI. [2]

Generative AI

- Creative
- Costly
- Hype ?

Predictive AI

- erministic
- Profitable

[1] McKinsey & Company, "The state of AI in 2025: Agents of innovation," McKinsey Global Institute, 2025.

[2] McKinsey & Company, "The state of AI: How organizations are rewiring to capture value," McKinsey Global Institute.

[3] Deloitte, "State of AI 2026 Global," Deloitte AI Institute, 2026.

The State of AI in 2026

Generative AI

- Creative
- Costly
- Hype ?

Predictive AI

- Deterministic
- Profitable
- Boring

The State of AI in 2026

Generative AI

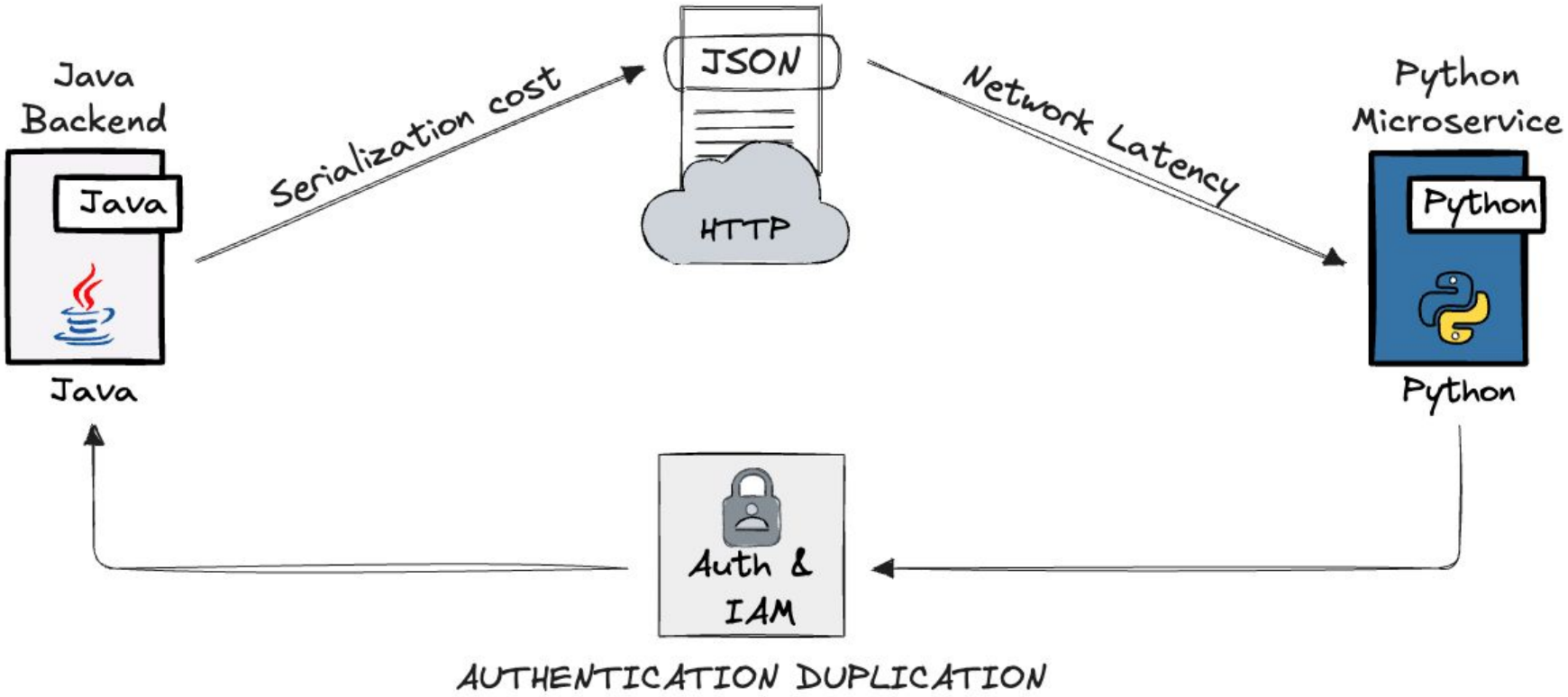
- Creative
- Costly
- Hype ?

Predictive AI

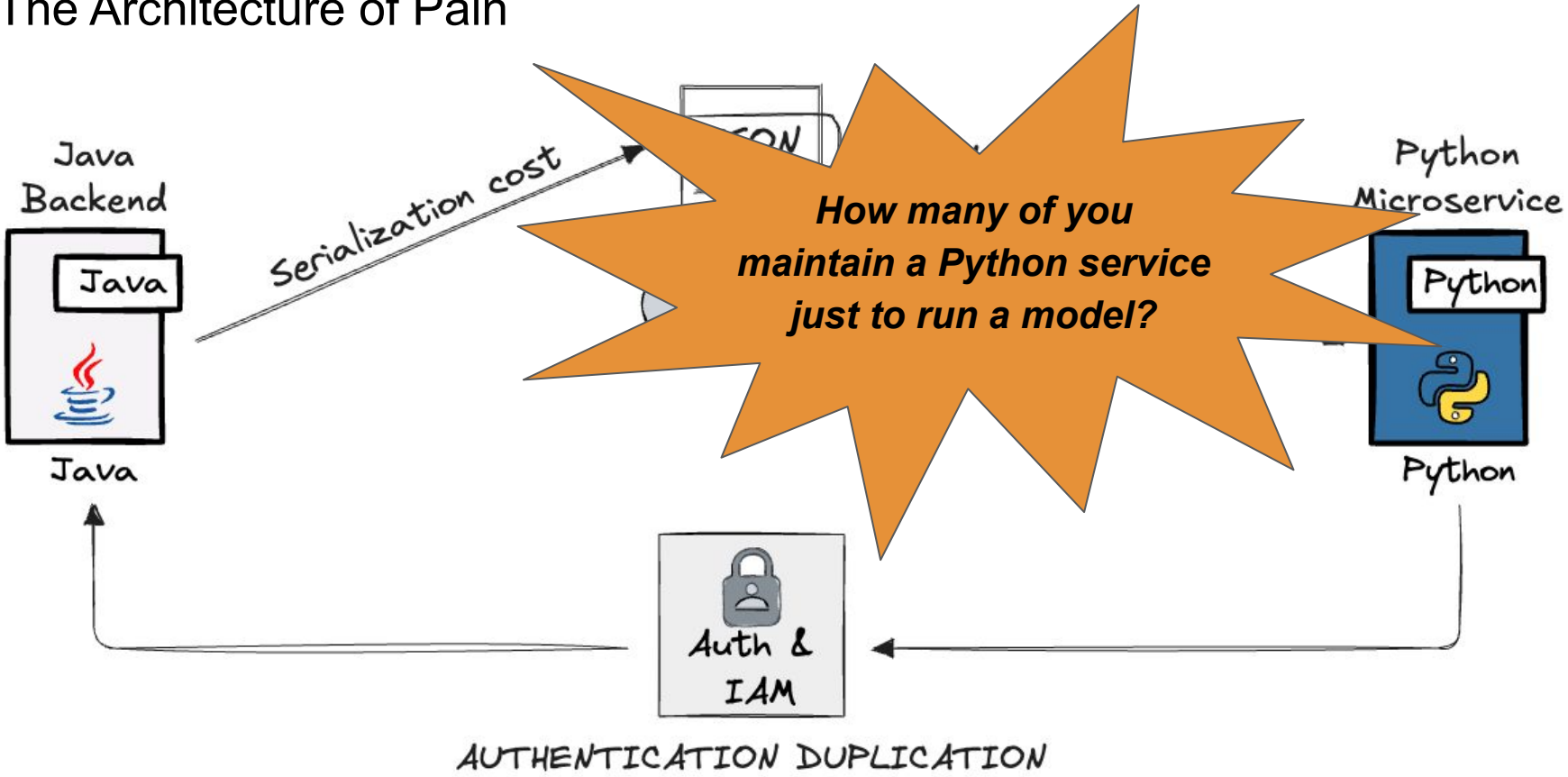
- Deterministic
- Profitable
- Boring

"GenAI is the artist; Predictive AI is the accountant. You need both, but the accountant runs the payroll"

The Architecture of Pain



The Architecture of Pain



The Thesis

Bring the Model to the Data, Not the Data to the Model.

Java owns the enterprise data.

Why are we shipping GBs of data to a Python container just to get a boolean isFraud = true?

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	
Inference		

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	
Inference		

[1] Oracle Labs, "Tribuo: Machine Learning in Java," 2020. [Online].
[2] SMILE — Statistical Machine Intelligence and Learning Engine, haifengl.github.io. [Online].
[3] Weka Wiki – Documentation, Waikato.github.io. [Online].
[4] E. Raff, "EdwardRaff/JSAT: Java Statistical Analysis Tool, a Java library for Machine Learning," GitHub. [Online].

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	
Inference	ONNX RUNTIME	

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	
Inference	ONNX RUNTIME	

[1] ONNX: Introduction to ONNX, ONNX.ai. [Online].

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	Spark MLlib The Machine Learning Library DL4J
Inference	ONNX RUNTIME	

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	Spark MLlib The Machine Learning Library DL4J
Inference	ONNX RUNTIME	

[1] MLlib — Apache Spark Machine Learning Library. Apache.org. [Online].
[2] Deeplearning4j — Deep Learning for the JVM. konduit.ai. [Online].

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	Spark MLlib The Machine Learning Library DL4J
Inference	ONNX RUNTIME	DJL DeepJavaLibrary ONNX RUNTIME PyTorch TensorFlow

The Java AI Landscape

	Classic ML / Tabular Data	Deep Learning / Big Data
Training	Tribuo JSAT	Spark MLlib The Machine Learning Library DL4J
Inference	ONNX RUNTIME	DJL DeepJavaLibrary → ONNX RUNTIME → PyTorch → TensorFlow

[1] Deep Java Library (DJL). djl.ai. [Online].

The Reality Check (Collaboration)

Training a model and serving a model at scale are two entirely different engineering domains.

- Your Data Scientists love PyTorch/Scikit-Learn.
- Do not force them to write Java.
- Let them train in Python.

The Reality Check (Collaboration)

Training a model and serving a model at scale are two entirely different engineering domains.

- Your Data Scientists love PyTorch/Scikit-Learn.
- Do not force them to write Java.
- Let them tra

Don’t rewrite.
Integrate

Business Data is Tabular

- **The "Boring" is the "Business":** Churn. Fraud. Credit. Demand. The biggest AI value lives in tabular data.
- **Java's Home Turf:** Fast. Accurate. Interpretable.

Introducing Tribuo (The Enterprise Choice)

1. Built-in Provenance

- **The Model's Birth Certificate:** Knows it data, knows its parameters, knows its transformations.
- **Verbatim Reproducibility:** Rebuild any production model. Exactly.

2. Native Type Safety

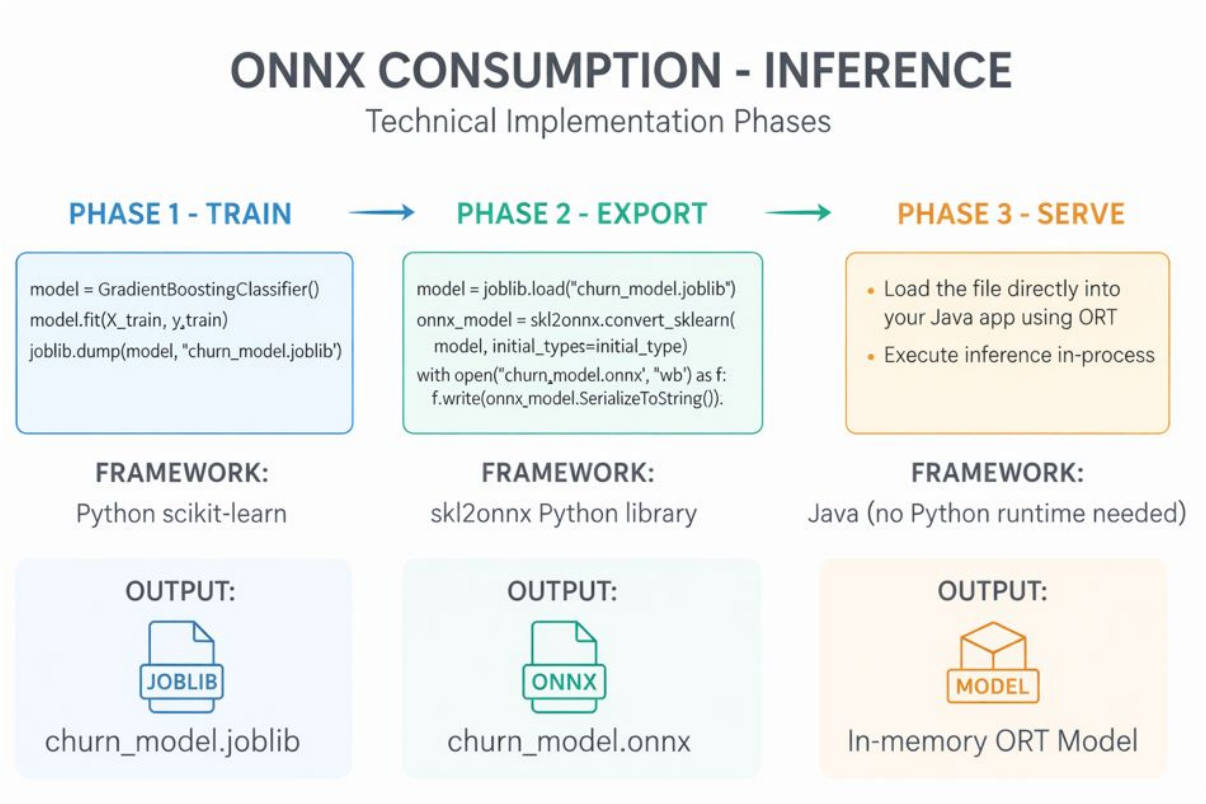
- **Shift-Left Errors:** Fail at compile time, not 2AM in production.
- **Schema Enforcement:** If the math and the code don't match, it won't load.

3. Seamless Interoperability

- **The Bridge:** TensorFlow. XGBoost. ONNX.
- **Hybrid Ecosystem:** Train in Python. Deploy in Java.

[1] Tribuo: Machine Learning in Java, Tribuo.org. [Online].

ONNX: The "PDF" of Machine Learning



[1] ONNX: Introduction to ONNX, ONNX.ai. [Online].

Case Study: Customer Churn Prediction

The Mission: Predict which customers will leave before they cancel.

- **The Business Stake:** Acquiring a new customer costs **5x–25x more** than retaining an existing one.
- **The Dataset:** Kaggle's Telco Churn (7,043 rows, 21 features).
- **Key Signals:**
  - **Tenure:** How long they've stayed.
  - **Contract:** Month-to-month vs. Multi-year.
  - **Support:** Tech support and online security status.
  - **Billing:** Monthly charges and payment method.
- **Target:** `Churn` (Yes/No).

CustomerID	Tenure	Contract	MonthlyCharges	Churn
7590-VHVEG	1	Month-to-month	$29.85	**No**
5575-GNVDE	34	One year	$56.95	**No**
3668-QPYBK	2	Month-to-month	$53.85	**Yes**

[1] Telco Customer Churn Dataset, Kaggle (blastchar). [Online].

Case Study: Customer Churn Prediction

Native Type Safety

- **The Problem:** In Python, a feature named `tenure` might be an `int`, a `float`, or a `None` at runtime. You only find out it's wrong when the model crashes in production.
- **The Java Solution:** Use Java Records or POJOs.

```
public record Customer(
    String id,
    String gender,
    float tenure,
    String contract,
    float monthlyCharges,
    // 16 other typed fields ...
) {}
```

Case Study: Customer Churn Prediction

Seamless Interoperability

- **The Problem:** Traditional ML requires "Model Microservices" (Flask/FastAPI), which introduce network latency and JSON serialization overhead.

- **The Java Solution:** Tribuo and ONNX Runtime live inside your JVM.

```java
public Prediction predict(Customer customer) {
    // 1. Map POJO to Native Tensors
    Map<String, OnnxTensor> inputs = mapToTensors(customer);
    // 2. Direct Memory Inference (via JNI)
    try (var results = session.run(inputs)) {
        float score = extractScore(results);
        return new Prediction(score > 0.5 ? "Yes" : "No");
    }
}
```

Case Study: Customer Churn Prediction

Provenance

- **The Problem:** Traditional ML models become orphans when exported. A black box you can't audit.
- **The Java Solution:** Tribuo treats **Provenance** as a first-class citizen. Every model ships with a **birth certificate**

```
{
  "className": "org.tribuo.ensemble.WeightedEnsembleModel",
  "datasetProvenance": {
    "numExamples": 4930,
    "numFeatures": 45,
    "tribuoVersion": "4.3.2"
  },
  "trainerProvenance": {
    "className": "RandomForestTrainer",
    "configuredParameters": {
      "numMembers": 50,
      "seed": 42,
      "maxDepth": 12
    }
  },
  "systemDetails": {
    "os": "Mac OS X",
    "javaVersion": "25.0.2",
    "arch": "aarch64"
  }
}
```

Case Study: Customer Churn Prediction

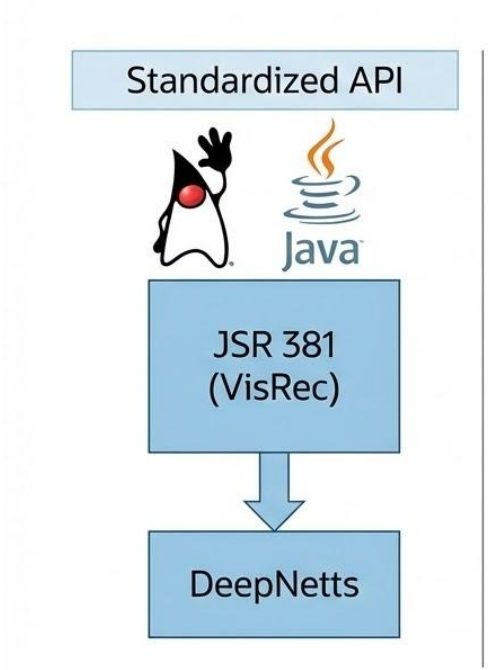
Performance

- **The Problem:** Remote API calls aren't free. Before the model even thinks, you pay for hundreds of milliseconds.
- **The Java Solution:** With ONNX, the model lives inside the JVM. Inference feels like a local function call.

Throughput (req/s)	Embedded (Java)	Remote (Python)
Avg	21.00	0.00723
Max	35.04	0.00893
Min	1.95	0.00083
p95	11.03	0.00600

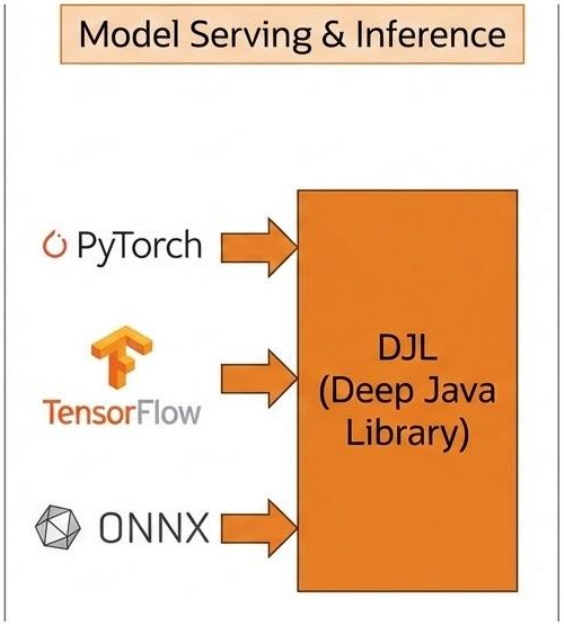
Latency (ms)	Embedded (Java)	Remote (Python)
Avg	0.047614	138.313
Min	0.028542	111.952
Max	0.512250	1206.54
p95	0.090666	166.729

The Deep Learning Landscape in Java



[1] Java Community Process, "JSR 381: Visual Recognition (VisRec) Specification"

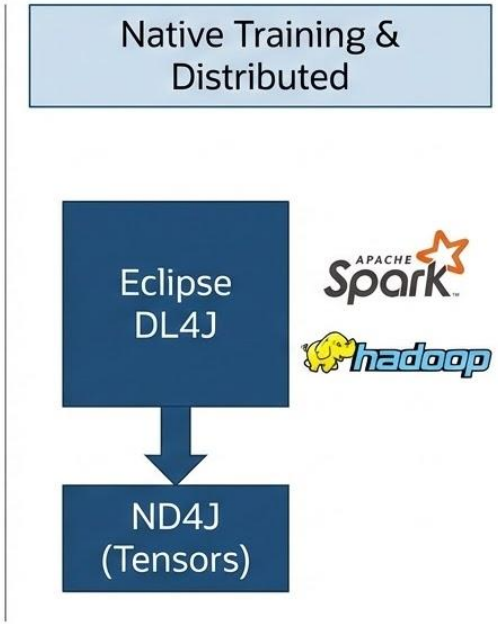
The Deep Learning Landscape in Java



[2] TensorFlow, “tensorflow/java: Java bindings for TensorFlow”

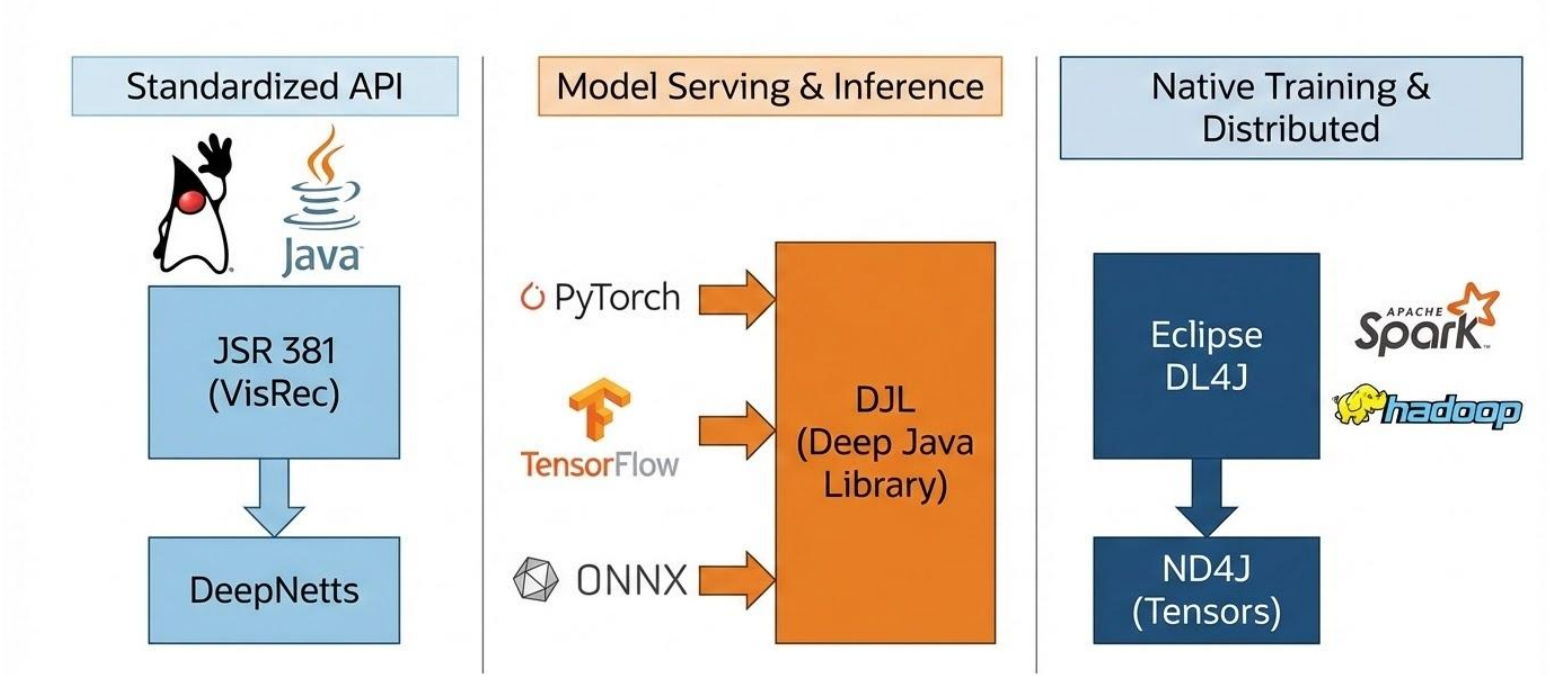
[3] Deep Java Library (DJL) Contributors, “Documentation — Deep Java Library”

The Deep Learning Landscape in Java



[4] Eclipse Deeplearning4j Community,

The Deep Learning Landscape in Java



[1] Java Community Process, "JSR 381: Visual Recognition (VisRec) Specification"

[2] TensorFlow, "tensorflow/java: Java bindings for TensorFlow"

[3] Deep Java Library (DJL) Contributors, "Documentation — Deep Java Library"

[4] Eclipse Deeplearning4j Community,

Real-World DJL: PyTorch to Java

- **Models:** YOLOv11 (.pt exported to .onnx).
- **Engine:** ONNX Runtime (via DJL).
- **Use Case:** Baseball → High-speed bat & ball tracking.

Models trained by: *Carlos Marcano* (@camarcano) and *Dylan Drummey* (@drummeydylan)

Real-World DJL: PyTorch to Java

- **Models:** YOLOv11 (.pt exported to .onnx).
- **Engine:** ONNX Runtime (via DJL).
- **Use Case:** Baseball → High-speed bat & ball tracking.

```java
Criteria<Image, DetectedObjects> criteria = Criteria.builder()
    .setTypes(Image.class, DetectedObjects.class)
    .optModelPath(modelDir)
    .optModelName("ball_tracking_v4-YOLOv11")
    .optTranslator(new BaseballDetectorTranslator())
    .optEngine("OnnxRuntime")
    .build();

ZooModel<Image, DetectedObjects> model  = criteria.loadModel();
Predictor<Image, DetectedObjects> predictor = model.newPredictor();
DetectedObjects detection = predictor.predict(img);
```

Inside the Translator

- The Translator interface provides model **pre-processing** and **postprocessing** functionality.

```java
class BatDetectorTranslator implements NoBatchifyTranslator<Image, DetectedObjects> {

    @Override
    public NDList processInput(TranslatorContext ctx, Image input) {
        // 1. Replicating Python's letterbox scaling & padding
        Image scaled = input.resize(newW, newH, true);
        // 2. Converting to CHW float array normalized to [0, 1]
        NDArray array = ctx.getNDManager().create(chwArray, new Shape(1, 3, 640, 640));
        return new NDList(array);
    }

    @Override
    public DetectedObjects processOutput(TranslatorContext ctx, NDList list) {
        // 1. Parsing raw tensor outputs
        float[] flatData = list.get(0).toFloatArray();
        // 2. Applying Non-Maximum Suppression (NMS) & IOU filtering
        return applyNMS(candidates, scores);
    }
}
```

Real-World DJL: PyTorch to Java

- **Models:** YOLOv11 (.pt exported to .onnx).
- **Engine:** ONNX Runtime (via DJL).
- **Use Case:** Baseball → High-speed bat & ball tracking.

```
Criteria<Image, DetectedObjects> criteria = Criteria.buil
    .setTypes(Image.class, DetectedObjects.class)
    .optModelPath(modelDir)
    .optModelName("ball_tracking_v4-YOLOv11")
    .optTranslator(new BaseballDetectorTranslator())
    .optEngine("OnnxRuntime")
    .build();

ZooModel<Image, DetectedObjects> model  = criteria.loadMo
Predictor<Image, DetectedObjects> predictor = model.newPr
DetectedObjects detection = predictor.predict(img);
```

Real-World DJL: PyTorch to Java - ***BENCHMARKING***

Performance

- **The Problem:** Remote API calls aren't free. Before the model even thinks, you pay for hundreds of milliseconds.
- **The Java Solution:** With ONNX, the model lives inside the JVM. Inference feels like a local function call.

Latency (ms)	Embedded (Java)	Remote (Python)
Avg	1487 ms	1702 ms
Min	1244 ms	1443 ms
Max	3173 ms	2967 ms
p95	2471 ms	2150 ms

Java 21+: The AI Hardware Accelerator

The Core Upgrades

- **Project Panama (Foreign Function & Memory API):**
  - **The "JNI Killer":** Safe, high-performance access to native memory and C++ libraries.
  - **Why it matters:** Direct, low-latency communication with ML kernels (like ONNX, TensorFlow, or CUDA) without the complex boilerplate or performance tax of JNI.

- **Vector API (Incubating):**
  - **SIMD Power:** Single Instruction, Multiple Data.
  - **Why it matters:** Allows the JVM to use specialized CPU instructions to process multiple data points in a single clock cycle, essential for the heavy matrix math found in Deep Learning.

But What About LLMs?

The Core Insight

- **LLMs are not Magic:** They are high-dimensional probability engines.
- **The RAG Bridge:** Retrieval Augmented Generation (RAG) is just **Search + Context**.
- **Vector Search is Classically Rooted:** "Vector Search" is essentially just a high-speed **K-Nearest Neighbors (K-NN)** algorithm.

Embeddings: The Universal Language

- **From Text to Math:** An embedding turns a "Customer Profile" or a "Product Description" into a vector (a list of numbers).
- **Semantic Similarity:** If two vectors are "close" in space, the concepts are related. This is how we build **Recommendation Engines** and **Semantic Search**.

DEMOS